Category A Assignment Problems

PROBLEM 1 • Basic → Intermediate

Membership Field Reach Checker

Problem Statement

PageTurner Library's IT team wants the same kind of pre-compile linter MediTrack built, but for their own LibraryMember class, with results broken down by modifier instead of a flat total. LibraryMember itself must also refuse (Week 1–4 string-validation and constructor skills) to be built at all with a blank or too-short membershipId.

Requirements

- classifyAccess(...) and summarizeByModifier(...) must decide their answers purely from the real Java visibility rules for SAME_CLASS, SAME_PACKAGE, and DIFFERENT_PACKAGE.
- Choose the right access level for each of LibraryMember's four fields, and give LibraryMember no usable no-argument constructor.
- LibraryMember's constructor must reject a membershipId that's blank, whitespace-only, or shorter than 4 characters.

Function Signature(s)

```
static String classifyAccess(String fieldModifier, String accessorContext)
static String summarizeByModifier(String[][] attempts)   // groups results per modifier
public LibraryMember(String membershipId, String branchCode, double finesOwed, String displayName)
```

Examples

Input	Output
`classifyAccess("private", "SAME_CLASS")`	`"ALLOWED"`

Input	Output
`classifyAccess("protected", "DIFFERENT_PACKAGE")`	`"DENIED"`

Input	Output
`summarizeByModifier({ {"private","SAME_CLASS"}, {"private","SAME_PACKAGE"}, {"default","SAME_PACKAGE"}, {"default","DIFFERENT_PACKAGE"}, {"protected","SAME_PACKAGE"}, {"protected","SAME_CLASS"}, {"public","DIFFERENT_PACKAGE"} })`	`"private: 1 allowed / 1 denied | default: 1 allowed / 1 denied | protected: 2 allowed / 0 denied | public: 1 allowed / 0 denied"`

Input	Output
`new LibraryMember("LB9", "BR1", 0, "Priya Nair")`	`construction rejected`

Explanation: *"LB9" is 3 characters — one short. Compare against "LB94", which should succeed.*

HINT Trim the string first, then check both blankness and minimum length in one pass — a value that's all spaces should fail exactly like an empty string.

CONSTRAINTS fieldModifier ∈ {"private","default","protected","public"}. accessorContext ∈ {"SAME_CLASS","SAME_PACKAGE","DIFFERENT_PACKAGE"}.

Key Concepts Tested: *String validation and parameterized constructors as a construction-time gate (Weeks 1–4), visibility rules, per-category aggregation.*

PROBLEM 2 • Intermediate

Reference Desk Subclass Reach

Problem Statement

PageTurner's premium-membership module (a different package) extends LibraryMember, and the reference desk keeps hitting confusing errors around finesOwed. Extend AccessChecker to handle the subclass-in-a-different-package rule, and add the same human-readable formatting touch (Week 1 string skills) MediTrack's engine got.

Requirements

- Add accessorContext values SUBCLASS_DIFFERENT_PACKAGE_OWN_TYPE and SUBCLASS_DIFFERENT_PACKAGE_PARENT_TYPE, and get protected's answer right for both.
- describeContext(...) must turn an underscore-separated code into a readable, title-cased sentence.

Function Signature(s)

```
static String classifyAccess(String fieldModifier, String accessorContext)
static String describeContext(String accessorContext)
```

Examples

Input	Output
`classifyAccess("protected", "SUBCLASS_DIFFERENT_PACKAGE_OWN_TYPE")`	`"ALLOWED"`

Input	Output
`classifyAccess("protected", "SUBCLASS_DIFFERENT_PACKAGE_PARENT_TYPE")`	`"DENIED"`

Explanation: *Java decides this from the compile-time type of the reference, not from what the object actually is at runtime.*

Input	Output
`describeContext("SUBCLASS_DIFFERENT_PACKAGE_OWN_TYPE")`	`"Subclass Different Package Own Type"`

HINT Ask "what's the declared type of the variable I'm accessing this through?" — not "what does this object actually happen to be right now?"

CONSTRAINTS Same modifier set as Problem 1, now with 5 possible accessorContext values.

Key Concepts Tested: *protected's subtle cross-package rule, compile-time vs. runtime typing, string splitting/case conversion (Week 1).*

PROBLEM 3 • Intermediate

Book Copy Circulation Guard

Problem Statement

A previous version of BookInventory once let its available-copy count go negative during a busy exam-week rush. Redesign it so the count can only move through validated methods — including a constructor that rejects an impossible starting inventory outright (Week 1–4 validation skills).

Requirements

- copiesTotal and copiesAvailable must both be private, with no way to set copiesAvailable directly from outside.
- The constructor must reject a copiesTotal that's zero or negative.
- checkOut() and checkIn() must each silently reject the transition that would push the count out of bounds.

Function Signature(s)

```
BookInventory(int copiesTotal)
void checkOut()
void checkIn()
int getCopiesAvailable()
```

Examples

Input	Output
`new BookInventory(0)`	`construction rejected`

Input	Output

`BookInventory b = new BookInventory(3);`
`b.checkOut(); b.checkOut(); b.checkOut();`
`b.checkOut();`
`b.getCopiesAvailable()`
	`0`

Explanation: *The 4th checkout has nothing left to take — rejected, count never goes negative.*

Input	Output

`b.checkIn(); b.checkIn(); b.checkIn();`
`b.checkIn();`
`b.getCopiesAvailable()`
	`3`

Explanation: *The 4th check-in has nothing genuine to return — the inventory is already full.*

HINT Think about what invariant must hold between copiesAvailable and copiesTotal at every single moment, not just at the two boundaries.

CONSTRAINTS copiesTotal, once valid, is a positive integer up to 500.

Key Concepts Tested: *Constructor-time validation (Weeks 1–4), private fields with validated methods, boundary-safe state transitions.*

PROBLEM 4 • Intermediate → Advanced

LibraryMember JavaBean, Chained Constructors & Security Answer

Problem Statement

PageTurner's kiosks use a JavaBean-scanning framework requiring a no-arg constructor, but signup also wants quicker constructors for partial data — chain all three with this(...) (Week 4). Add a security-answer property that's write-only for good.

Requirements

- Build three constructors — no-arg, name-only, and id+name — chained together with this(...).
- Every ordinary field needs a JavaBean-compliant getX()/setX() pair (isPremiumMember(), not getPremiumMember()); setMembershipId(...) must only take effect once, ever.
- securityAnswer must be settable but never retrievable again, in any form.

Function Signature(s)

```
public LibraryMember()
public LibraryMember(String name)
public LibraryMember(String membershipId, String name)
String getMembershipId();  void setMembershipId(String id)
boolean isPremiumMember();  void setPremiumMember(boolean premium)
void setSecurityAnswer(String answer)   // no matching getter exists anywhere
```

Examples

Input	Output
`new LibraryMember("Priya Nair").getMembershipId()`	`null`

Input	Output
`new LibraryMember("LIB-8841", "Priya Nair").getMembershipId()`	`"LIB-8841"`

Input	Output

`LibraryMember m = new LibraryMember();`
`m.setMembershipId("LIB-8841");`
`m.setMembershipId("FAKE-0000");`
`m.getMembershipId()`
	`"LIB-8841"`

Explanation: *The second call is silently ignored — write-once, on top of a setter the framework requires to exist.*

HINT Route the no-arg and name-only constructors through this(...) into the same one real initialization path, instead of repeating field assignments three times.

CONSTRAINTS securityAnswer's stored transformation only needs to be deterministic and one-way.

Key Concepts Tested: *Constructor chaining with this(...) (Week 4), JavaBean getX/setX/isX naming, a write-once property, a true write-only property.*

PROBLEM 5 • Advanced

Immutable Loan Receipt & Nightly Circulation Ledger

Problem Statement

Every checkout issues a LoanReceipt that must never change once created. Build it as genuinely immutable, validating each book ID's format (Week 1 string skills) at construction, then reconcile a night's worth of receipts — including a reference-only variant — without ever crashing on a null entry.

Requirements

- LoanReceipt's fields must all be final, the class itself final, and every book ID must match the format "BK-" followed by exactly 3 digits — reject the whole array otherwise.
- bookIds must be defensively copied on the way in and the way out; correcting a receipt means returning a brand-new object via a with-style method.
- The nightly processor must use instanceof to settle a ReferenceOnlyLoanReceipt differently from a regular one, and must never throw on a null batch entry. Set up any one-time shared state with a static block.

Function Signature(s)

```
public LoanReceipt(String memberId, String[] bookIds)
String[] getBookIds()
LoanReceipt withCorrectedBookId(int index, String newId)
public ReferenceOnlyLoanReceipt(String memberId, String[] bookIds, String roomNumber)
static String processNightlyCirculation(LoanReceipt[] receipts)
```

Examples

Input	Output
`new LoanReceipt("LIB-8841", new String[]{"BK-100", "bad"})`	`construction rejected`

Explanation: *"bad" doesn't match "BK-" + 3 digits — the whole construction fails.*

Input	Output

`LoanReceipt r = new LoanReceipt("LIB-8841", new String[]{"BK-100","BK-101"});`
`String[] ids = r.getBookIds();`
`ids[0] = "HACKED";`
`r.getBookIds()[0]`
	`"BK-100"`

Input	Output

`processNightlyCirculation({`
`  new ReferenceOnlyLoanReceipt("LIB-001", new String[]{"BK-200"}, "Reading Room 3"),`
`  null,`
`  new LoanReceipt("LIB-002", new String[]{"BK-201"})`
`})`
	`"2 processed | 1 null skipped | 1 reference-only | 1 regular"`

HINT Every array field needs defending in both directions independently — a correct constructor copy says nothing about whether the getter copies too.

CONSTRAINTS bookIds length up to 20. Batch size up to 5,000.

Key Concepts Tested: *String format validation (Week 1), true immutability with defensive copying, the "wither" pattern, static blocks, instanceof dispatch, null-safety — everything from Weeks 1–6 combined.*